

PGRC — Piattaforma per la Gestione di Ricette di Cucina

Relazione di progetto — Programmazione Web e Mobile, A.A. 2025/2026

Autore: **Lorenzo Tricarico** — matricola: **70598A**

Modalità: **Progetto Full**

1. Introduzione	2
2. Analisi dei requisiti	2
3. Architettura dell'applicazione	3
3.1 Applicazione multi-pagina	3
3.2 Organizzazione del codice JavaScript	3
3.3 Presentazione	4
4. La sorgente di informazioni: TheMealDB e Web Storage	5
4.1 Acquisizione dei dati (API REST)	5
4.2 Modello dei dati nel Web Storage	5
5. Struttura e presentazione delle pagine	7
5.1 Struttura comune	7
5.2 Le pagine	7
5.3 Presentazione (CSS)	8
6. Realizzazione delle operazioni richieste	8
6.1 Gestione del profilo utente	8
6.2 Ricerca delle ricette	9
6.3 Scheda della ricetta	10
6.4 Ricettario personale e note private	11
6.5 Recensioni	11
7. Riepilogo delle scelte implementative	11
8. Prove di funzionamento	12
9. Conclusioni e possibili estensioni	17

1. Introduzione

PGRC è un portale web per la gestione di ricette culinarie: un utente registrato può cercare ricette (i cui dati provengono dalle API REST di [TheMealDB](https://www.themealdb.com/)), aggiungerle al proprio **ricettario personale**, personalizzarle, annotarle con **note private** e pubblicare **recensioni** visibili a tutti gli utenti.

Come richiesto dalla traccia, il lavoro ha seguito cinque fasi:

1. analisi dei requisiti;
2. identificazione delle funzionalità da sviluppare;
3. progettazione della struttura e della presentazione delle pagine web;
4. progettazione della sorgente di informazioni;
5. implementazione.

La specifica è per sua natura incompleta e ambigua: nel corso della relazione, e in particolare nel punto 7, sono documentate le scelte interpretative fatte e le relative motivazioni.

Il progetto è sviluppato in modalità **Full**: oltre alle funzionalità core del progetto Light (registrazione/login, gestione dei dati utente, ricettario, ricerca per titolo, visualizzazione delle ricette) sono implementate la ricerca per ingredienti e per testo descrittivo, la scheda ricetta completa di recensioni, le note private e l'inserimento/rimozione delle recensioni.

2. Analisi dei requisiti

Dalla traccia sono stati individuati quattro macro-scenari:

1. **Gestione del profilo utente** — registrazione (con username, email, password e piatti preferiti), login/logout, modifica dei dati personali, cancellazione del profilo. Alla registrazione va creato automaticamente un ricettario personale vuoto.
2. **Ricerca di ricette** — ricerca sequenziale (sfoglia) o per parole chiave: nome del piatto, ingredienti, testo descrittivo, lettera iniziale. I dati delle ricette (elenco, ingredienti e ogni altra informazione) devono essere acquisiti tramite le **API REST di TheMealDB**.
3. **Gestione del ricettario personale** — aggiunta e rimozione delle ricette dal ricettario tramite un pulsante nella scheda della ricetta; possibilità di associare a ciascuna ricetta una nota testuale **privata** (non visibile agli altri utenti).
4. **Recensioni** — ogni utente può recensire una ricetta specificando data di preparazione, voto di difficoltà (1–5) e voto di gusto (1–5); le recensioni sono pubbliche e vanno mostrate nella scheda della ricetta.

Vincoli tecnici espliciti: solo **HTML5**, **CSS3** e **JavaScript** con separazione

tra struttura (HTML) e presentazione (CSS); allo startup tutti i dati necessari devono essere **scaricati dalle API di TheMealDB in formato JSON, memorizzati nel Web Storage** del browser e visualizzati nell'applicazione; devono esistere operazioni di presentazione e modifica delle informazioni nel Web Storage.

Un punto ambiguo della specifica Light («gestione dei dati dell'utente ... sia utente finale sia ristoratore») è stato interpretato come refuso proveniente da un'altra traccia: in PGRC non esiste alcuna funzione riservata a un "ristoratore", quindi è stato modellato **un solo tipo di utente registrato**.

3. Architettura dell'applicazione

3.1 Applicazione multi-pagina

L'applicazione è **multi-pagina**: una pagina HTML per ogni vista (`index.html`, `search.html`, `recipe.html`, `cookbook.html`, `profile.html`, `login.html`, `register.html`, `reset.html`). La scelta, alternativa alla single-page application, è motivata da:

- aderenza al paradigma "classico" HTML5+CSS3+JS richiesto dal corso, senza router artificiali in JavaScript;
- URL significativi e navigabili (`recipe.html?id=...`, `search.html?type=...&q=...`), che rendono ogni vista raggiungibile e ricaricabile direttamente;
- semplicità: ogni pagina carica solo lo script che le serve.

Lo stato condiviso tra le pagine (utenti, sessione, ricette, recensioni) vive nel Web Storage, che per sua natura sopravvive alla navigazione.

3.2 Organizzazione del codice JavaScript

Il codice è **JavaScript vanilla** (nessun framework) ed è organizzato a livelli, ciascuno in un file con una responsabilità precisa:

File	Responsabilità
js/storage.js	Persistenza: wrapper <code>Storage</code> su <code>localStorage</code> (JSON) e oggetto <code>DB</code> con le operazioni sui dati applicativi
js/api.js	Integrazione con le API REST di TheMealDB (download e normalizzazione) e funzioni di ricerca sui dati locali

js/auth.js	Registrazione, login/logout, modifica/cancellazione profilo, recupero password
js/ui.js	Livello di interfaccia condiviso: utility, componenti riusabili, rendering, azioni globali
js/pages/*.js	Uno script per pagina: definisce la vista e i suoi eventi

Il flusso dei dati è sempre: **pagina** → **`Auth`/`API` (logica)** → **`DB` (persistenza)** → **`Storage` (localStorage)**.

Ogni script di pagina definisce un oggetto globale ``Page`` con un contratto uniforme:

- ``render(params)`` — restituisce l'HTML della vista (``params`` è un ``URLSearchParams`` con la query string);
- ``bind(params)`` — (facoltativo) aggancia gli eventi ai form appena inseriti;
- ``requiresAuth: true`` — (facoltativo) la pagina richiede il login.

All'evento ``DOMContentLoaded``, ``ui.js`` esegue per ogni pagina la stessa sequenza: guardia di autenticazione (redirect a ``login.html`` per le pagine riservate), **seeding** dei dati da TheMealDB (§4), quindi ``render()`` della vista dentro `<main id="view">`. Dopo ogni azione che modifica i dati la vista viene ridisegnata richiamando ``render()``, senza ricaricare la pagina.

Le azioni trasversali (aggiungi/rimuovi dal ricettario, elimina nota, rimuovi recensione, cancella account, "Mostra altri", ...) sono bottoni con attributo ``data-action``, gestiti da **un unico listener globale con event delegation**: il listener sul ``document`` funziona anche per l'HTML rigenerato dopo un ``render()``, evitando di riagganciare gli handler a ogni ridisegno.

3.3 Presentazione

La presentazione è interamente delegata ai fogli di stile: **Bootstrap 5** (via CDN) fornisce reset, griglia e componenti di base, mentre l'aspetto è definito dal tema personalizzato in ``css/style.css``, caricato dopo Bootstrap (§5.3). Nessuna regola di stile è scritta negli attributi HTML: la separazione struttura/presentazione richiesta dalla traccia è così rispettata.

4. La sorgente di informazioni: TheMealDB e Web Storage

4.1 Acquisizione dei dati (API REST)

La traccia impone che allo startup tutti i dati necessari siano scaricati dalle API di TheMealDB, memorizzati nel Web Storage e visualizzati. L'implementazione (`API.seedIfNeeded` in `js/api.js`) segue esattamente questo modello:

1. al primo avvio l'applicazione esegue **26 richieste GET** all'endpoint REST `https://www.themealdb.com/api/json/v1/1/search.php?f=<lettera>`, una per ogni lettera a–z, scaricando l'intero dataset pubblico di TheMealDB (ricette complete di ingredienti, dosi, immagine, procedimento, categoria, area geografica, tag e link YouTube);
2. le richieste sono eseguite **in parallelo** con `Promise.allSettled`, così il fallimento di una singola lettera non blocca le altre; una callback di avanzamento aggiorna il messaggio di caricamento (``n/26``);
3. ogni risposta JSON viene **normalizzata** (`API.normalizeMeal`): il formato di TheMealDB, che rappresenta gli ingredienti in 20 coppie di campi piatti (``strIngredient1..20` / `strMeasure1..20``), viene convertito in un modello compatto con un array ``ingredients: [{name, measure}]``;
4. il risultato è salvato nel Web Storage e il completamento del seeding viene marcato con un timestamp (`pgrc_seeded_at`): alle visite successive il download non viene ripetuto e l'app lavora sui dati locali.

Se il download fallisce e non esiste una cache, la pagina mostra una schermata di errore con il pulsante "Riprova"; se esiste una cache anche parziale, l'applicazione resta utilizzabile.

Scelta implementativa. TheMealDB espone anche endpoint di ricerca puntuali (`search.php?s=`, `filter.php?i=`, ...); si è scelto di **non** interrogarli a ogni ricerca ma di scaricare l'intero dataset una sola volta, perché: (a) è il modello esplicitamente richiesto dalla traccia ("allo startup, tutti i dati necessari ... memorizzati nel web storage"); (b) elimina la latenza di rete e le chiamate ripetute all'API; (c) permette ricerche non offerte dall'API remota, come la ricerca sul testo del procedimento o quella combinata su tutti i campi.

4.2 Modello dei dati nel Web Storage

Tutti i dati sono memorizzati in `localStorage` in **formato JSON**, tramite il wrapper `Storage` (`read`/`write`/`remove`` con `JSON.parse`/`JSON.stringify`` e

gestione degli errori di parsing). Le chiavi e i relativi schemi:

```
pgrc_users : [{ id, username, email, passwordHash, salt, favoriteDishes[],
  security: { question, answerHash },
  cookbook: [{ recipeld, addedAt,
    custom: { name?, instructions? } | null,
    notes: [{ id, text, createdAt }] }],
  createdAt }]
pgrc_session : { userId, loginAt } | null
pgrc_recipes : { "<idMeal>": { id, name, category, area, instructions, thumb,
  tags[], youtube, ingredients: [{name, measure}] } }
pgrc_reviews : [{ id, recipeld, userId, username, prepDate,
  difficulty (1-5), taste (1-5), comment, createdAt }]
pgrc_seeded_at: timestamp dell'ultimo download completo
```

Scelte di modellazione principali:

- **Ricette in una mappa `id` → `ricetta`** anziché in un array: l'accesso per id (scheda ricetta, voci del ricettario) è diretto, e l'inserimento dal seeding (`upsertRecipes`) è idempotente. L'elenco ordinato per nome è derivato al volo da `getRecipeList()`.
- **Ricettario e note dentro l'oggetto utente**: ricettario e note sono dati privati del singolo utente, quindi vivono nel suo `cookbook`; questo rende naturale il requisito di privacy delle note (un altro utente non le vede perché non fanno parte dei suoi dati) e fa sì che la cancellazione dell'account elimini automaticamente ricettario e note.
- **Recensioni in una collezione separata** (`pgrc\_reviews`): sono dati pubblici, letti per ricetta (`getReviewsForRecipe`) e devono sopravvivere alla navigazione di qualunque utente; ogni recensione denormalizza lo `username` dell'autore per la visualizzazione (mantenuto coerente quando l'utente cambia username).
- **Sessione come chiave dedicata** (`pgrc\_session`): si è scelto `localStorage` anche per la sessione, così il login sopravvive alla chiusura del browser (comportamento "resta collegato"). `sessionStorage` è usato solo per i messaggi **flash** tra una pagina e l'altra (§5.2).

Le operazioni di **presentazione e modifica** delle informazioni richieste dalla traccia sono implementate nell'oggetto `DB` (utenti, sessione, ricette, recensioni: lettura, inserimento, aggiornamento, cancellazione) e usate da tutte le pagine.

5. Struttura e presentazione delle pagine

5.1 Struttura comune

Tutte le pagine condividono la stessa struttura HTML5 semantica:

- `<header class="site-header">` con il **brand** e la barra di navigazione
`<nav id="main-nav">`; le voci hanno l'attributo `data-auth="user"` o `data-auth="guest"` e vengono mostrate/nasconde in base allo stato di login (es. "Il mio ricettario" e "Profilo" solo per gli utenti autenticati, "Accedi"/"Registrati" solo per gli ospiti); la voce attiva è evidenziata confrontando `data-nav` con `data-page` sul `<body>`;
- `<main id="view">`, il contenitore in cui `render()` inserisce la vista generata da `Page.render(params)`; all'apertura contiene lo spinner di caricamento del seeding;
- `<footer class="site-footer">` con i crediti (fonte dati TheMealDB e nota sulla persistenza nel Web Storage);
- un elemento `#toast` (`role="status"`, `aria-live="polite"`) per i messaggi di feedback.

Ogni pagina carica gli script comuni (`storage.js`, `api.js`, `auth.js`, `ui.js`) più il proprio script in `js/pages/`.

5.2 Le pagine

Pagina	Contenuto
<code>index.html</code> (Home)	Hero con messaggio personalizzato per l'utente loggato, form di ricerca rapida, barra delle lettere a-z e anteprima delle prime ricette
<code>search.html</code>	Form di ricerca (tipo + termine), barra a-z, risultati in griglia con conteggio e paginazione "Mostra altri"
<code>recipe.html?id=...</code>	Scheda completa della ricetta: immagine, categoria/area, tag, medie dei voti, pulsante aggiungi/rimuovi dal ricettario, ingredienti con dosi, procedimento, sezione note private e sezione recensioni
<code>cookbook.html</code>	Ricettario personale (richiede login): elenco delle ricette salvate con eventuale personalizzazione e note

`profile.html`	Profilo (richiede login): riepilogo dati, form di modifica con cambio password opzionale, cancellazione account
`login.html`	Accesso con username o email + password
`register.html`	Registrazione completa (credenziali, piatti preferiti, domanda e risposta di sicurezza)
`reset.html`	Recupero password in due passi tramite domanda di sicurezza

Essendo l'applicazione multi-pagina, un messaggio di conferma mostrato subito prima di una navigazione andrebbe perso con il ricaricamento: per questo esiste il meccanismo dei messaggi **flash**, salvati in `sessionStorage` e mostrati come toast all'apertura della pagina successiva (es. "Benvenuto!" dopo il login).

5.3 Presentazione (CSS)

`css/style.css` è l'unico foglio di stile del progetto ed è organizzato in tre sezioni:

1. **variabili** (`:root`): palette (brand arancio, inchiostro, carta), raggio dei bordi, ombre e font di sistema — modificando le variabili si ritematizza l'intera applicazione;
2. **tema dei componenti**: header/nav, hero, form di ricerca, barra delle lettere, card e griglia delle ricette, bottoni, scheda ricetta, note, recensioni, ricettario, profilo, toast; le classi omonime a quelle di Bootstrap (` .btn`, ` .card`, ` .form-control`, ...) **ridefiniscono** il componente Bootstrap corrispondente;
3. **livello di compatibilità Bootstrap** (in coda): ripristina i margini tipografici azzerati dal **reboot** e normalizza campi form e stati focus, così la resa resta identica al design originale.

Il layout è **responsive**: la griglia delle ricette si adatta alla larghezza disponibile e due breakpoint (720px e 420px) riorganizzano header, form e griglie per tablet e smartphone. Sono curati anche gli aspetti di accessibilità di base: HTML semantico, `label` sui campi, attributi `aria-\*` su toast, barra delle lettere e form di ricerca, `alt` sulle immagini.

6. Realizzazione delle operazioni richieste

6.1 Gestione del profilo utente

Registrazione (`register.html`, `Auth.register`). Il form raccoglie username, email, password (con conferma), piatti preferiti (stringa separata da virgole, convertita in array) e una domanda di sicurezza con risposta, usata per il recupero password. La validazione controlla lunghezze minime (username ≥ 3 , password ≥ 6), formato dell'email, coincidenza delle password e **unicità** di username ed email (confronto case-insensitive). Alla registrazione viene creato l'utente con **ricettario personale vuoto** (`cookbook: []`, come richiesto dalla traccia) e la sessione viene aperta automaticamente.

Sicurezza delle credenziali. Non essendoci un backend, le password non possono uscire dal browser; si è comunque scelto di **non salvarle mai in chiaro**: per ogni utente viene generato un `salt` casuale e nel Web Storage finisce solo l'hash **SHA-256** di `salt + password`, calcolato con la Web Crypto API (`crypto.subtle.digest`). Lo stesso schema protegge la risposta di sicurezza (normalizzata con trim+minuscole prima dell'hash, così il confronto non è sensibile a maiuscole o spazi). Il login ricalcola l'hash e lo confronta con quello memorizzato.

Login/Logout (`login.html`, `Auth.login`/`Auth.logout`). L'accesso accetta indistintamente username o email; a login riuscito viene scritta la sessione `{userId, loginAt}`. Il logout elimina la sessione. Le pagine riservate (`cookbook.html`, `profile.html`) dichiarano `requiresAuth: true` e vengono protette dalla guardia di autenticazione, che reindirizza al login gli ospiti.

Modifica dei dati (`profile.html`, `Auth.updateProfile`). L'utente può modificare username, email e piatti preferiti; il cambio password è opzionale e richiede la password attuale. La validazione replica quella della registrazione e verifica l'unicità di username/email **rispetto agli altri account**. Se lo username cambia, viene aggiornato anche nelle recensioni già pubblicate, per mantenerle coerenti.

Cancellazione dell'account (`Auth.deleteAccount`). Previa conferma (`confirm`), l'utente viene rimosso insieme a tutte le sue recensioni; la sessione viene chiusa e si torna alla home. Ricettario e note, essendo contenuti nell'oggetto utente, vengono eliminati con esso.

Recupero password (`reset.html`). In due passi: dalla email si ottiene la domanda di sicurezza dell'account; con la risposta corretta (verificata per hash) si imposta la nuova password.

6.2 Ricerca delle ricette

Tutte le ricerche operano sui dati locali nel Web Storage (§4.1) e sono raggiungibili da `search.html?type=...&q=...`; il form di ricerca è presente anche nella home. Le modalità implementate:

`type`	Funzione	Comportamento
`name`	`searchByName`	Ricette il cui nome contiene il termine (case-insensitive)
`ingredient`	`searchByIngredient`	Ricette con almeno un ingrediente che contiene il termine
`text`	`searchByText`	Ricerca sul testo descrittivo : procedimento, categoria, area geografica e tag
`letter`	`searchByLetter`	Ricette il cui nome inizia con la lettera scelta (barra a–z)
`all`	`searchAll`	Ricerca combinata su tutti i campi; con termine vuoto mostra l'intero elenco (ricerca sequenziale/sfoglia)

La "ricerca sequenziale" richiesta dalla traccia corrisponde alla modalità `all` senza termine (l'intero catalogo ordinato alfabeticamente, sfogliabile) e alla barra delle lettere. I risultati sono mostrati in una griglia di card (immagine, nome, categoria · area) con conteggio dei risultati e **paginazione incrementale**: vengono renderizzate 24 card alla volta e il pulsante "Mostra altri" ne aggiunge altre 24, per non appesantire il DOM con centinaia di nodi.

6.3 Scheda della ricetta

`recipe.html?id=...` recupera la ricetta dal Web Storage e ne mostra tutte le informazioni principali richieste: **immagine**, categoria, area, tag, **ingredienti con le dosi** e **procedimento**. Completano la scheda:

- le **medie dei voti** (gusto e difficoltà) calcolate dalle recensioni;
- il pulsante "**Aggiungi al ricettario**" / "**Rimuovi dal ricettario**", che per gli ospiti reindirizza al login;

- la sezione **note private** (§6.4);
- la sezione **recensioni** (§6.5), visibile a tutti.

Il caso di id inesistente (ricetta non trovata) è gestito con una vista dedicata.

6.4 Ricettario personale e note private

Il ricettario è creato vuoto alla registrazione e popolato dalla scheda della ricetta. Ogni voce del ricettario (``cookbook.html``) mostra la ricetta salvata e offre:

- **rimozione** dal ricettario (con conferma: si perdono anche le note);
- **personalizzazione** di titolo e procedimento (``entry.custom``): l'utente può riscrivere la ricetta "a modo suo" senza toccare l'originale condiviso nella cache; un badge segnala le ricette personalizzate ed è possibile ripristinare la versione originale. Questa è l'interpretazione data al requisito Light "modifica ... delle sue ricette": si modifica la **propria copia** nel ricettario, non il dato condiviso;
- le **note testuali private**: dalla scheda della ricetta l'utente può aggiungere note (``{id, text, createdAt}``) ed eliminarle. Le note vivono nel ``cookbook`` dell'utente, quindi sono strutturalmente invisibili agli altri utenti, come richiesto; per aggiungere una nota la ricetta deve prima essere nel ricettario (la nota è un'annotazione della **propria** copia).

6.5 Recensioni

Dalla scheda della ricetta ogni utente autenticato può pubblicare una recensione con i tre campi richiesti dalla traccia — **data di preparazione** (campo ``date``, limitato a oggi come massimo), **voto di difficoltà (1–5)** e **voto di gusto (1–5)** — più un commento facoltativo. Le recensioni sono pubbliche: la scheda le mostra a tutti (anche agli ospiti), ordinate dalla più recente, con i voti resi come stelle. Ogni utente può **rimuovere soltanto le proprie** recensioni (il pulsante compare solo sulle proprie e la cancellazione verifica l'autore anche a livello di ``DB.deleteReview``).

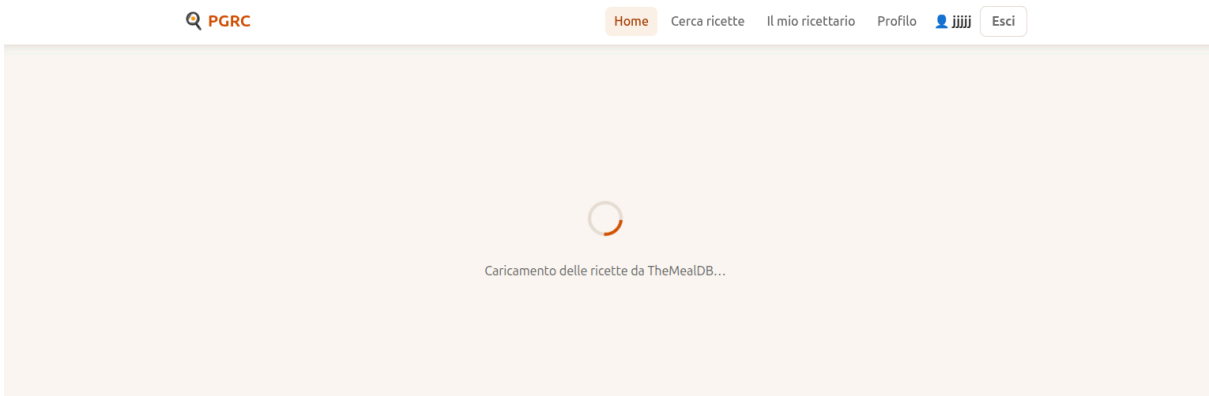
7. Riepilogo delle scelte implementative

Scelta	Motivazione
Applicazione multi-pagina	URL significativi, niente router JS, una vista = una pagina; lo stato condiviso vive nel Web Storage
Download dell'intero dataset allo startup, ricerche locali	Modello imposto dalla traccia; niente latenza né chiamate ripetute; abilita ricerche non offerte dall'API (testo del procedimento, combinata)
Ricette in mappa id → ricetta	Accesso diretto per id e seeding idempotente
Ricettario e note dentro l'oggetto utente	Privacy delle note garantita per costruzione; cancellazione account = cancellazione dati
Recensioni in collezione separata	Dati pubblici, interrogati per ricetta, indipendenti dall'utente che naviga
Password con hash SHA-256 + salt (Web Crypto)	Mai credenziali in chiaro nel Web Storage
Recupero password con domanda di sicurezza	Non esiste un backend che possa inviare email
Pattern Page.render/bind + event delegation	Contratto uniforme tra le pagine; gli handler sopravvivono ai ridisegni della vista
Escape sistematico (esc()) di ogni dato interpolato	Prevenzione di HTML/script injection dai contenuti inseriti dagli utenti (note, recensioni, username)
Promise.allSettled nel seeding	Il fallimento di una lettera non compromette il download delle altre
Bootstrap 5 + tema personalizzato	Componenti e griglia consolidati, ma aspetto interamente controllato da css/style.css
Messaggi flash in sessionStorage	Feedback che sopravvive alla navigazione tra pagine
Paginazione "Mostra altri" (24 card)	Il dataset completo (~300 ricette) non viene mai renderizzato in blocco
Un solo tipo di utente (niente "ristoratore")	Il riferimento al ristorante nella specifica Light è stato valutato come refuso: nessuno scenario di PGRC gli attribuisce funzioni

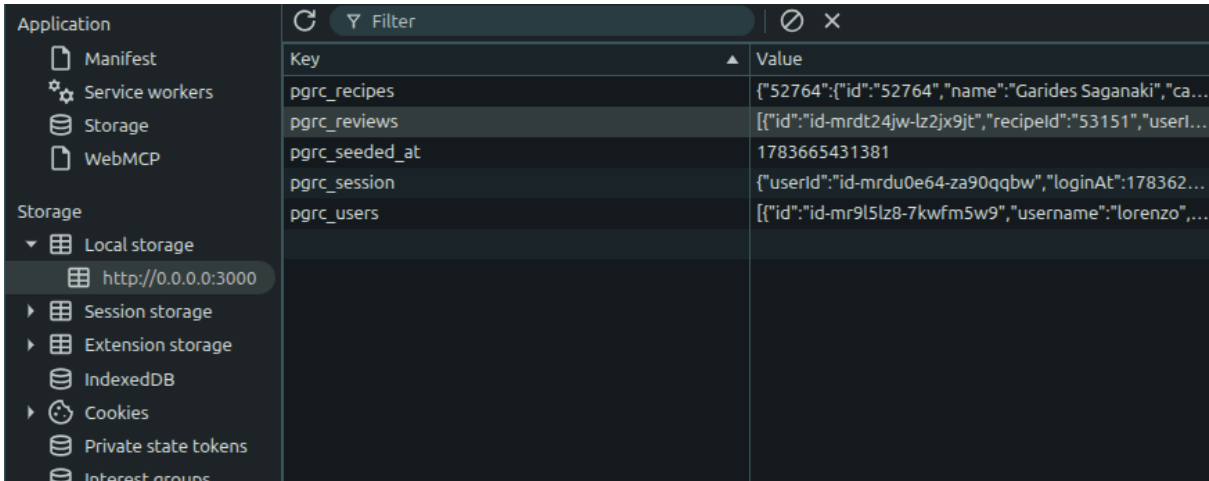
8. Prove di funzionamento

Le schermate dimostrative sono raccolte di seguito, una per ciascuna operazione prevista.

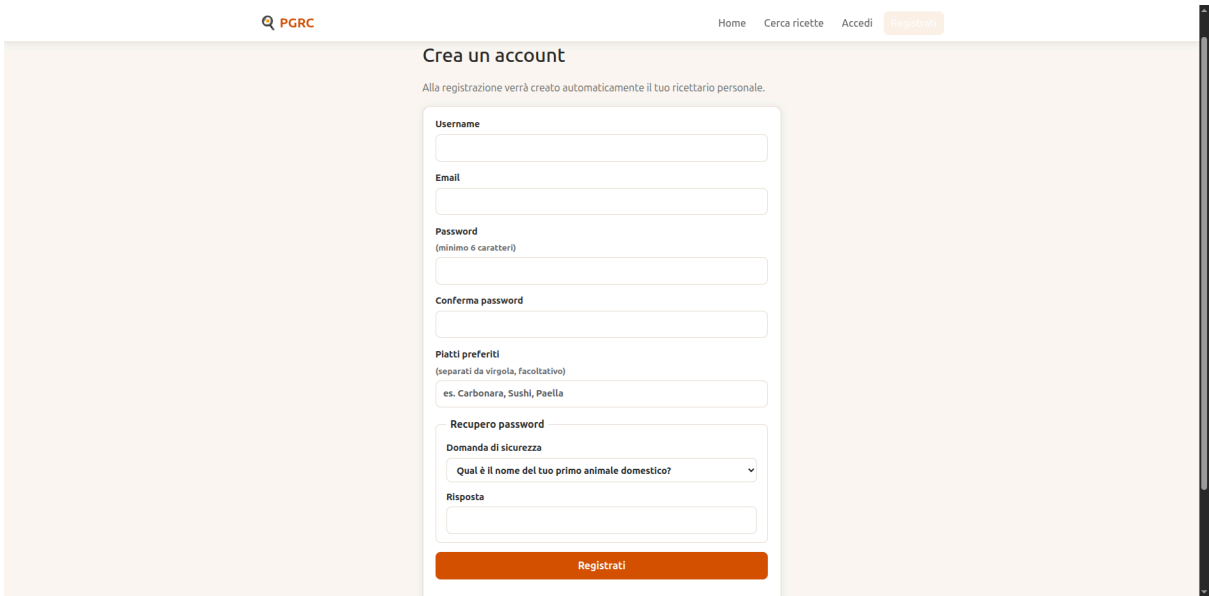
1. Primo avvio: caricamento del dataset da TheMealDB (spinner con avanzamento n/26) e toast di conferma del salvataggio nel Web Storage.



2. Web Storage popolato (DevTools → Application → Local Storage: chiavi `pgrc\_\*`).



3. Registrazione: form compilato, validazione (es. password troppo corta, email già in uso) e accesso automatico dopo la registrazione.



Crea un account

Alla registrazione verrà creato automaticamente il tuo ricettario personale.

Username

X

Please lengthen this text to 3 characters or more (you are currently using 1 character).

X@X.X

Password
(minimo 6 caratteri)

...

Conferma password

...

Piatti preferiti
(separati da virgola, facoltativo)

es. Carbonara, Sushi, Paella

Crea un account

Alla registrazione verrà creato automaticamente il tuo ricettario personale.

Username

XXX

Email

X@X.X

Password
(minimo 6 caratteri)

...

Please lengthen this text to 6 characters or more (you are currently using 3 characters).

...

Piatti preferiti
(separati da virgola, facoltativo)

4. Login e logout (anche con username al posto dell'email); tentativo di accesso a `cookbook.html` da ospite con redirect al login.

Accedi

Username o email

Password

Accedi

[Password dimenticata?](#) · Non hai un account? [Registrati](#)

Video di redirect attraverso Middleware se si prova ad accedere ad url protette:

Una registrazione del comportamento è disponibile al link [seguente](#)

Tutorial completo [piattaforma](#)

Tutti gli asset (screenshot e video) sono inclusi nel repository consegnato.

5. Ricerca per titolo, per ingrediente, per testo descrittivo, per lettera iniziale e combinata ("tutti i campi"), con conteggio dei risultati e pulsante "Mostra altri".

Cerca ricette

Titolo del piatto

g

Cerca

A B C D E F **G** H I J K L M N O P Q R S T U V W X Y Z

Ricette che iniziano per "G"

16 risultati



Gambas al ajillo
Seafood · Spanish



Garides Saganaki
Seafood · Greek



Garlicky prawns with sherry
Seafood · Spanish



General Tsos Chicken
Chicken · Chinese



Cerca ricette

Tutti i campi

cook

Cerca

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

Risultati per "cook" in tutti i campi

490 risultati



Acaraje black-eyed pea fritters with shrimp filling
Seafood



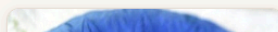
Adana kebab
Lamb · Turkish



Air Fryer Egg Rolls
Side · Chinese



Air fryer patatas bravas
Vegetarian · Spanish



6. Scheda della ricetta: informazioni complete, medie dei voti, pulsante aggiungi/rimuovi dal ricettario.



Paella

Seafood - Cucina Spanish

Gusto ★★★★★ 4.5/5
Difficoltà ★★☆☆☆ 2.0/5

+ Aggiungi al ricettario
▶ Video ricetta su YouTube

Ingredienti

Olive Oil	3 tablespoons
Raw tiger prawns	10
Parsley	Bunch
Dry sherry	100 ml
Mussels	500g
Saffron	Pinch
Chorizo	150g
Onion	1 chopped
Garlic	3 cloves
Squid	1 medium

Procedimento

- step 1
- Heat 1 tbsp of the oil in a wide, shallow pan. Add the prawn heads and parsley stalks and sizzle until the heads turn pink, then mash with a potato masher. Pour over the sherry or wine and 300ml water, season with salt and simmer for 10 mins to make a stock, mashing the prawn heads as they cook.
- step 2
- Scatter the mussels into the pan, cover the pan loosely with a lid or tea towel, then put over a high heat for 3-4 mins until the mussels just open. Stir to release the mussel juices, then pour the contents of the pan into a colander set over a large bowl containing the saffron. Let the saffron steep in the stock – you will need 700ml in total, so top up with water if needed and give everything a good stir. Pick the mussels out from the colander, then set aside.
- step 3

Note personali

Aggiungi la ricetta al tuo ricettario per inserire note private.

Recensioni (2) Difficoltà ★★☆☆☆ | Gusto ★★★★★

- fff ha preparato il piatto il 8 luglio 2026

Gusto: ★★★★★ Difficoltà: ★☆☆☆☆

facile e buono
- lorenzo ha preparato il piatto il 8 luglio 2026

Gusto: ★★★★★ Difficoltà: ★★★★★

fd0skd0ifjds
- Rimuovi

Lascia una recensione

Data di preparazione

Voto difficoltà

Voto gusto

mm/dd/yyyy

3 — ★★★★★

4 — ★★★★★

Commento (facoltativo)...

Pubblica recensione

7. Ricettario: elenco delle ricette salvate, personalizzazione di titolo e procedimento con badge e ripristino dell'originale, rimozione di una ricetta.

[Home](#) [Cerca ricette](#) [Il mio ricettario](#) [Profilo](#)

fff

[Esci](#)

Il mio ricettario

2 ricette salvate. Puoi personalizzarle, annotarle o rimuoverle.

Algerian Bouzgene Berber Bread with Roasted Pepper Sauce

Side · Algerian · aggiunta il 10 luglio 2026

[Apri scheda](#) [Modifica](#) [Rimuovi](#)

Titolo personalizzato

Procedimento personalizzato

Questo è il mio procedimento

Salva modifiche

Ripristina originale

Paella

Seafood · Spanish · aggiunta il 11 luglio 2026

[Apri scheda](#) [Modifica](#) [Rimuovi](#)

Il mio ricettario

2 ricette salvate. Puoi personalizzarle, annotarle o rimuoverle.



Nome custom piatto personalizzata

Side - Algerian - aggiunta il 10 luglio 2026

Procedimento personalizzato: Questo è il mio procedimento

[Apri scheda](#)[Modifica](#)[Rimuovi](#)

Paella

Seafood - Spanish - aggiunta il 11 luglio 2026

[Apri scheda](#)[Modifica](#)[Rimuovi](#)

8. Note private: inserimento e rimozione; verifica con un secondo utente che le note non sono visibili ad altri.

Il mio ricettario

1 ricetta salvata. Puoi personalizzarle, annotarle o rimuoverle.



Paella jjj personalizzata

Seafood · Spanish · aggiunta il 9 luglio 2026

Procedimento personalizzato: vlpdvp

Apri scheda

Modifica

Rimuovi

9. Recensioni: inserimento con data di preparazione e voti 1–5, visualizzazione pubblica nella scheda, rimozione della propria recensione (e assenza del pulsante sulle recensioni altrui).

Recensioni (3) Difficoltà ★★☆☆☆ | Gusto ★★★★★

jjjjj ha preparato il piatto il 30 giugno 2026

[Rimuovi](#)

Gusto: ★★☆☆☆ Difficoltà: ★★★★★

Recensione mia di quando l'ho preparato.

fff ha preparato il piatto il 8 luglio 2026

Gusto: ★★★★★ Difficoltà: ★☆☆☆☆

facile e buono

lorenzo ha preparato il piatto il 8 luglio 2026

Gusto: ★★★★★ Difficoltà: ★☆☆☆☆

fd0skd0ifjdsd

Lascia una recensione

Data di preparazione

Voto difficoltà

Voto gusto

mm/dd/yyyy



3 — ★★☆☆☆



4 — ★★★★★



Commento (facoltativo)...

[Pubblica recensione](#)

10. Profilo: modifica dei dati (con cambio password), recupero password con domanda di sicurezza, cancellazione dell'account con rimozione delle sue recensioni.



HomeCerca ricetteIl mio ricettarioProfiloEsci

Modifica dati personali

Username

Email

Piatti preferiti
(separati da virgola)

Cambio password (facoltativo)

Password attuale

Nuova password

Conferma nuova password

Salva modifiche

Cancellazione

La cancellazione del profilo rimuove definitivamente account, ricettario,

11. Layout responsive (viewport mobile).
Tablet

Piattaforma per la Gestione di Ricette di Cucina

Cerca tra **733** ricette, salva le tue preferite nel ricettario personale e lascia le tue recensioni.

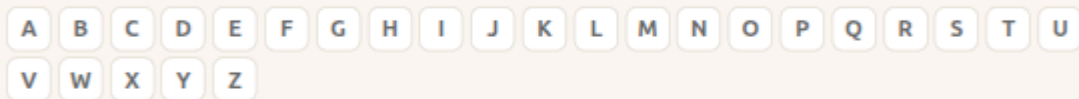
Titolo del piatto

Cerca una ricetta...

Cerca

Non hai un account? [Registrati gratis.](#)

Sfoglia per lettera



Alcune ricette

[Vedi tutte](#)



Acaraje black-eyed pea fritters with shrimp filling
Seafood



Adana kebab
Lamb - Turkish



Air Fryer Egg Rolls
Side - Chinese



Air fryer patatas bravas
Vegetarian - Spanish



Ajo blanco
Starter - Spanish



Alfajores
Dessert - Argentina

Smartphone

Piattaforma per la Gestione di Ricette di Cucina

Cerca tra **733** ricette, salva le tue preferite nel ricettario personale e lascia le tue recensioni.

Titolo del piatto



Cerca una ricetta...

Cerca

Non hai un account? [Registrati gratis.](#)

Sfoglia per lettera

A

B

C

D

E

F

G

H

I

J

K

L

M

N

O

P

Q

R

S

T

U

V

W

X

Y

Z

Alcune ricette

[Vedi tutte →](#)



9. Conclusioni e possibili estensioni

Il progetto implementa tutte le funzionalità del progetto **Full** (e quindi anche quelle **Light**), rispettando i vincoli tecnici della traccia: HTML5 con separazione struttura/presentazione, dati acquisiti dalle API REST di TheMealDB allo startup, persistenza in formato JSON nel Web Storage e operazioni di presentazione e modifica dei dati memorizzati. Oltre ai requisiti sono state aggiunte alcune funzionalità extra: recupero password con domanda di sicurezza, hash delle credenziali, personalizzazione delle ricette del ricettario, paginazione dei risultati e toast di feedback.

Possibili estensioni future: aggiornamento periodico del dataset (re-seeding dopo una scadenza), filtri combinabili (categoria + area + ingrediente), ordinamenti dei risultati (per voto medio), esportazione del ricettario in JSON e modifica delle recensioni già pubblicate.